

RetinalCare

AI-assisted diabetic retinopathy screening & remote clinical review

Technical Pitch Deck

A healthcare platform connecting retinal imaging, AI-assisted analysis, remote doctor review, and patient access.

Architecture • AI pipeline • Technology stack • Safety • Implementation

1. Problem & Objective

Problem

Patients in remote or underserved regions may have retinal images captured without easy access to an ophthalmologist. This can delay specialist assessment and create unnecessary travel.

Objective

RetinalCare connects retinal imaging with remote healthcare professionals and AI-assisted analysis so that images can be screened and reviewed through a structured workflow.

Technical goal	Clinical workflow goal
Provide an integrated frontend, FastAPI backend, database, and AI inference workflow.	Keep AI as decision support while a doctor performs the final assessment.

2. System Architecture

The platform separates the user interfaces, healthcare API, persistent data, and AI analysis components.

Doctor Review & assessment	Technical Staff Patient & scan workflow	Patient Portal & results
	↓	
React Frontend Role-specific interfaces		
↓		
FastAPI Backend Authentication • Patients • Scans • Doctors • Dashboard • Chat		
	↓	
PostgreSQL Healthcare data	AI Inference PyTorch / DR models / Grad-CAM	Gemini Assistant Educational knowledge workflow

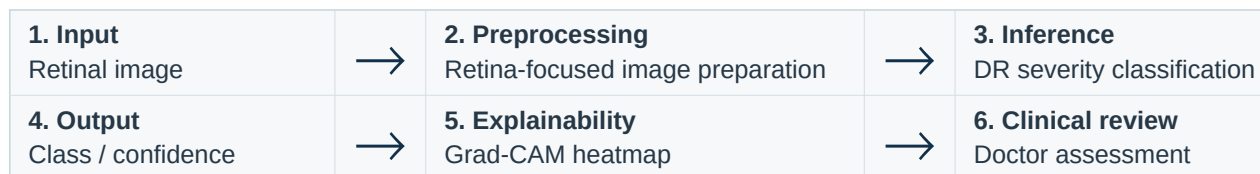
3. Technology Stack

Layer	Technology / component	Role
Frontend	React / TypeScript	Role-specific healthcare dashboards and portals
Backend	FastAPI / Python	REST API, authentication, patient/scan/doctor workflows
Data	PostgreSQL + SQLAlchemy	Persistent healthcare application data
Authentication	JWT	Authenticated access and role-aware API usage
Computer Vision	PyTorch • EfficientNet • Siamese network	Retinal image classification and learned feature extraction
Explainability	Grad-CAM	Visual indication of image regions influencing model output
Educational AI	Gemini + diabetes_info.pdf / knowledge files	Educational responses about diabetes and eye health

Design principle: keep the AI layer separated from the clinical workflow so model output can assist, rather than silently replace, professional review.

4. AI / Computer-Vision Pipeline

The computer-vision project contains multiple model approaches and an explainability component.



Documented model work in the CV project

- Simple CNN and several transfer-learning / vision architectures are documented.
- EfficientNet is used for retinal feature extraction.
- A Siamese network combines information from two eye branches using EfficientNetB3-based components.
- Grad-CAM is used to produce heatmaps for visual interpretation.

5. Technical Choices & Trade-offs

Choice	Why it fits	Trade-off / consideration
FastAPI	Lightweight Python API suited to integrating ML inference.	Requires disciplined API validation and production configuration.
PostgreSQL	Structured relational storage for users, patients, scans and assessments.	Adds database setup compared with a local-only prototype.
EfficientNet / Siamese	Strong image-feature extraction and two-eye modeling in the documented CV work.	More computation and deployment complexity than a small CNN.
Grad-CAM	Adds an interpretable visual layer to model predictions.	A heatmap explains model attention, not clinical causality.
Gemini educational assistant	Provides a conversational interface over curated diabetes/eye-health material.	Requires API-key management and safeguards against unsupported medical claims.

6. End-to-End Healthcare Workflow

The application implements separate role-oriented workflows while keeping the doctor as the final reviewer.

Technical Staff	AI / Backend	Doctor	Patient
Register / find patient Upload retinal scan			
Submit case	Store scan → run AI analysis → generate result / heatmap		
	Make AI output available for review	Review image + AI result + heatmap Add assessment / notes	
Receive review status	Persist final assessment	Submit final assessment	Access permitted results / recommendations

7. Educational AI Assistant

The platform includes an educational assistant intended to answer questions about diabetes, diabetic retinopathy, and eye health.

- Gemini is integrated through the backend rather than exposing the API key in the frontend.
- `backend/diabetes\_info.pdf` and the backend knowledge resources provide the documented information base.
- The assistant is positioned as educational support and should not replace a clinician or provide a definitive diagnosis.
- API credentials are kept in environment configuration and should never be committed to the public repository.

Key implementation files

Backend: `backend/routes/chat.py`, `backend/config.py`, `backend/knowledge/`, `backend/diabetes\_info.pdf`
Frontend: `frontend/src/components/EducationalChatWidget.tsx` and its CSS

8. Security, Privacy & Clinical Safety

- Authentication uses JWT-based access for protected application endpoints.
- The Gemini API key is supplied through environment configuration rather than frontend source code.
- Local database files, uploads, logs, and large ML artifacts are excluded from Git tracking.
- Patient workflows are separated from doctor and technical-staff workflows.
- AI output is decision support; the doctor remains the final reviewer in the intended clinical workflow.
- For real deployment, privacy, consent, access control, audit logging, secure transport, and clinical validation require further production hardening.

Important boundary

This project demonstrates a healthcare software and AI-assisted screening workflow. Model outputs and educational AI responses should not be treated as a standalone medical diagnosis.

9. Repository Structure & Local Run

Where to look first

README.md	Project overview and orientation
backend/	FastAPI application, routes, database, authentication and AI/chat integration
frontend/	React/TypeScript healthcare interfaces
CS502-diabetic-retinopathy-detection/	Computer-vision project, model code, notebooks and dataset documentation
CS502.../data/README.md	Dataset source and reproducibility documentation

Local run

```
Backend: from `backend/`, run `uvicorn main:app --reload`.
Frontend: from `frontend/`, run `npm start`.
Configure required environment variables (including database and Gemini credentials) locally; do not commit `.env`.
```

10. Current Technical Position

RetinalCare combines a healthcare application layer with an existing retinal computer-vision project and an educational AI assistant.

Core integration

- Role-based healthcare frontend connected to a FastAPI backend.
- Persistent healthcare data through PostgreSQL / SQLAlchemy.
- Retinal image analysis using the documented PyTorch model work.
- Grad-CAM visualization to support model interpretation.
- Doctor review and final assessment workflow.
- Patient portal and technical-staff scan workflow.
- Educational Gemini assistant backed by project knowledge resources.

Repository: github.com/firas55-maker/diabetic-retinopathy-healthcare

Thank you